

5.3.4. Автоматные грамматики

Линейные грамматики (праворекурсивные и леворекурсивные) называются *автоматными грамматиками*, так как языки, порождаемые ими, совпадают с языками, распознаваемыми конечными автоматами.

Теорема. Для каждой праволинейной грамматики существует эквивалентный конечный автомат.

Доказательство. Каждому нетерминалу произвольной праволинейной грамматики G поставим в соответствие одно состояние конечного автомата A . Добавим еще одно состояние - единственное конечное состояние. Состояние, соответствующее аксиоме, назовем начальным состоянием.

Каждому правилу $A \rightarrow aB$ поставим в соответствие команду $Aa \rightarrow B$, а каждому терминальному правилу $A \rightarrow a$ поставим в соответствие команду $Aa \rightarrow F$.

Таким образом, выводу цепочки в грамматике

$S \Rightarrow a_1A_1 \Rightarrow a_1a_2A_2 \Rightarrow \dots \Rightarrow a_1a_2 \dots a_{k-1}A_{k-1} \Rightarrow a_1a_2 \dots a_k$ взаимно однозначно соответствует последовательность команд построенного автомата A :

$$Aa_1 \rightarrow A_1; A_1a_2 \rightarrow A_2; \dots; A_{k-1}a_k \rightarrow F.$$

Таким образом, язык $L(G) = L(A)$.

Пример. Пусть для заданной грамматики G требуется построить конечный автомат.

$$S \rightarrow aS \mid bB;$$

$$A \rightarrow aA \mid bS;$$

$$B \rightarrow bB \mid c \mid cA.$$

Решение. Граф автомата будет иметь четыре вершины, три из них помечены нетерминальными символами грамматики S, A, B , четвертая вершина, помеченная символом F , является единственным заключительным состоянием. Начальным состоянием является вершина, соответствующая аксиоме S .

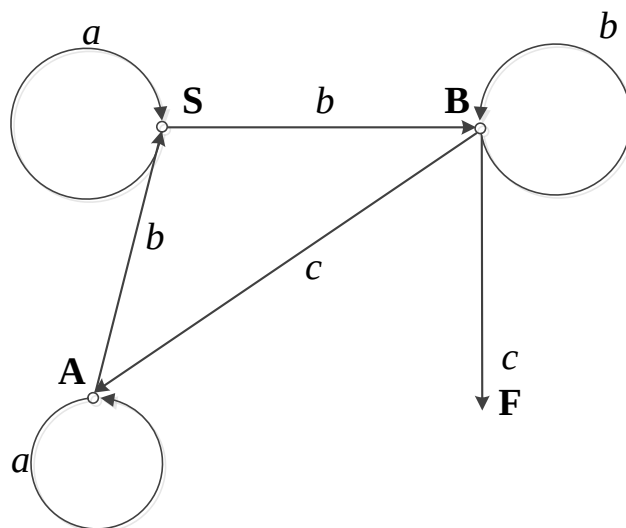


Рис. 9. Граф конечного автомата

Каждому правилу грамматики поставим в соответствие команду конечного автомата:

$Sa \rightarrow S$ - в начальном состоянии при поступлении на вход терминального символа a автомат остается в том же состоянии S ;

$Sb \rightarrow B$ - из начального состояния при поступлении на вход терминала b автомат переходит в состояние B ;

$Bb \rightarrow B$ - в состоянии B при поступлении на вход терминала b автомат остается в том же состоянии B ;

$Bc \rightarrow F$ - из состояния B при поступлении на вход терминала c автомат переходит в заключительное состояние F ;

$Bc \rightarrow A$ - из состояния B при поступлении на вход терминала c автомат переходит в состояние A ;

$Aa \rightarrow A$ - в состоянии A при поступлении на вход терминала a автомат остается в этом же состоянии A ;

$Ab \rightarrow S$ - из состояния A при поступлении на вход терминала b автомат переходит в состояние S .

Полученный недетерминированный конечный автомат распознает цепочки языка, порождаемые праворекурсивной грамматикой G .

Теорема. Для произвольного конечного автомата существует эквивалентная праволинейная грамматика.

Доказательство. Каждому состоянию произвольного автомата поставлен в соответствие нетерминал грамматики, причем начальному состоянию будет соответствовать аксиома.

Тогда для каждой команды $Ac \rightarrow Bb$ множество правил грамматики включим правило $A \rightarrow cB$, причем в случае, если B - заключительное состояние, добавим правило $A \rightarrow c$.

Исходная грамматика и построенный конечный автомат эквивалентны.

Теорема. Для каждой леворекурсивной грамматики существует эквивалентный конечный автомат.

Доказательство. Каждому нетерминальному символу произвольной леворекурсивной грамматики поставим в соответствие состояние конечного автомата, причем состояние, соответствующее аксиоме S , сделаем заключительным. Добавим еще одно состояние N и сделаем его начальным состоянием.

Каждому правилу грамматики $A \rightarrow Ba$ поставим в соответствие команду автомата $Ba \rightarrow A$. Тогда каждому выводу в грамматике:

$S \Rightarrow A_1 a_k \Rightarrow A_2 a_{k-1} a_k \Rightarrow \dots \Rightarrow A_{k-1} a_2 a_3 \dots a_k \Rightarrow a_1 a_2 \dots a_k$ однозначно соответствует следующая последовательность команд построенного автомата A :

$$Na_1 \rightarrow A_{k-1}; \dots; A_2 a_{k-1} \rightarrow A_1; A_1 a_k \rightarrow S.$$

Таким образом, язык $L(G) = L(A)$.

Любую автоматную грамматику можно представить в виде соответствующего конечного графа. По графу грамматики легко отыскивается вывод нужной цепочки. Любой вывод цепочки в автоматной грамматике соответствует пути в графе этой грамматики, который начинается из вершины S (вершины, помеченной аксиомой) и заканчивается в конечной вершине.

Пример. Построить конечный автомат, распознающий язык $L(A) = \{(ab)^*\}$.

Сначала построим некоторую грамматику G , которая бы порождала язык $L(A)$:

$$\begin{aligned} S &\rightarrow aA; \\ A &\rightarrow bS \mid b. \end{aligned}$$

Проверим, действительно ли эта грамматика порождает язык $L(A)$. Для этого построим несколько выводов возможных вариантов цепочек:

- 1) $S \Rightarrow aA \Rightarrow ab$;
- 2) $S \Rightarrow aA \Rightarrow abS \Rightarrow abaA \Rightarrow abab$;
- 3) $S \Rightarrow aA \Rightarrow abS \Rightarrow abaA \Rightarrow ababS \Rightarrow ababaA \Rightarrow ababab$; и т.д.

Таким образом, грамматика G действительно порождает язык $L(A)$, следовательно, можно построить соответствующий этой грамматике конечный автомат. Для этого введем заключительное состояние F , начальное состояние соответствует аксиоме S .

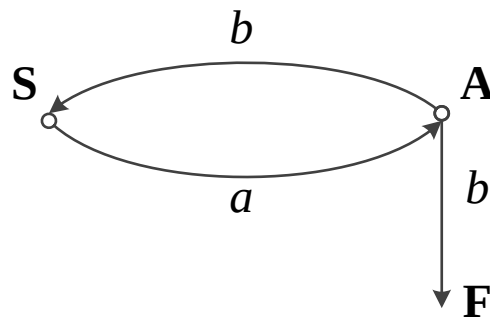


Рис. 9. Граф конечного автомата, распознающего язык $L(A) = \{(ab)^*\}$

Запишем преобразование правил вывода в команды:

$Sa \rightarrow A$ - из состояния S при поступлении на вход терминала a автомат переходит в состояние A ;

$Ab \rightarrow S$ - из состояния A при поступлении на вход терминала a автомат переходит в состояние S ;

$Ab \rightarrow F$ - из состояния A при поступлении на вход терминала b автомат переходит в заключительное состояние F .

Таким образом, построен недетерминированный конечный автомат, распознающий заданный язык $L(G)$.

5.4. Автоматы с магазинной памятью

Автомат с магазинной памятью (МП-автомат) имеет рабочую ленту, которая организована в виде магазина.

МП-автомат - это семерка вида:

$M = (K, \Sigma, \Gamma, \delta, p_0, F, B_0)$, где

K - конечное множество состояний;

Σ - алфавит;

Γ - алфавит магазина;

δ - функция переходов;

p_0 - начальное состояние;

F - множество заключительных состояний;

B_0 - символ из множества Γ для обозначения маркера дна магазина.

В общем случае данное определение соответствует недетерминированному автомату.

В отличие от конечного автомата для произвольного МП-автомата нельзя построить эквивалентный детерминированный автомат.

Основное использование распознавательных средств задания языков состоит в построении алгоритмов *грамматического разбора*. Поэтому необходимо для произвольной КС-грамматики уметь строить эквивалентный МП-автомат.

МП-автомат представляет интерес как средство разбора в КС-грамматиках произвольного вида. Этот факт сформулирован в следующей теореме.

Теорема. Языки, порождаемые КС-грамматиками, совпадают с языками, распознаваемыми МП-автоматами.

Доказательство. Существуют две стратегии разбора: *восходящий* и *нисходящий* разбор.

5.4.1. Восходящий разбор в МП-автомате

При восходящей стратегии необходимо найти основу и редуцировать ее к какому-нибудь нетерминалу в соответствии с правилами данной грамматики. Это можно сделать в том случае, если реализовать следующий алгоритм функционирования МП-автомата:

- а) любой входной символ записывается в магазин;
- б) если в верхушке магазина сформирована основа, совпадающая с правой частью правила, то она заменяется на нетерминал в левой части этого правила;
- с) разбор заканчивается, если в магазине остается аксиома, а входная цепочка рассмотрена полностью.

В соответствии с этим алгоритмом для КС-грамматики $G = (V^T, V_N, P, S)$ построим МП-автомат:

$M = (K, V_T, \Gamma, \delta, p_0, F, B_0)$, где $\Gamma = V_T \cup V_N \cup \{B_0\}$,

$K = \{p_0, F\}$, $F = \{f\}$.

Функция переходов δ будет содержать следующие команды:

- а) $p_0, a, \varepsilon \rightarrow p_0, a$ - для любых $a \in V_T$;
- б) $p_0, \varepsilon, \phi' \rightarrow p_0, A$ - для всех правил $A \rightarrow \phi \in P$, где ϕ' - зеркальное отображение ϕ ;
- с) $p_0, \varepsilon, SB_0 \rightarrow f, B_0$.

В общем случае команда выглядит так:

$p_i, \sigma, \gamma \rightarrow p_j, \lambda$, где $p_i \in K$ – состояние автомата до выполнения команды, $\sigma \in V_t$ – символ на входной ленте, $\gamma \in \Gamma$ – символ верхушки магазина, $p_j \in K$ – состояние автомата после выполнения команды, $\lambda \in \Gamma$ – символ, который записывается в магазин.

Таким образом, любому выводу в грамматике G взаимно однозначно соответствует последовательность команд построенного МП-автомата. Обратное построение КС-грамматики по произвольному МП-автомату также возможно, но не представляет практического интереса.

Пример. Пусть дана грамматика G :

$S \rightarrow S+A \mid S/A \mid A$

$A \rightarrow a \mid (S);$

$$V_N = \{S, A\}, V_T = \{a, (,), +, /\}.$$

Для заданной КС-грамматики G необходимо построить МП-автомат при помощи восходящей стратегии разбора. Эквивалентный МП-автомат должен содержать следующие команды:

1. Команды переноса терминальных символов в магазин:

$$p_0, a, \varepsilon \rightarrow p_0, a;$$

$$p_0, +, \varepsilon \rightarrow p_0, +;$$

$$p_0, /, \varepsilon \rightarrow p_0, /;$$

$$p_0, (\varepsilon \rightarrow p_0, (;$$

$$p_0, \varepsilon \rightarrow p_0, \varepsilon).$$

Эти команды обеспечивают занесение терминального символа из входной ленты в магазин.

2. Команды редукции по правилам грамматики:

$$p_0, \varepsilon, A+S \rightarrow p_0, S;$$

$$p_0, \varepsilon, A/S \rightarrow p_0, S;$$

$$p_0, \varepsilon, A \rightarrow p_0, S;$$

$$p_0, \varepsilon,)S(\rightarrow p_0, A;$$

$$p_0, \varepsilon, a \rightarrow p_0, A.$$

Эти команды заменяют зеркальное отображение правила, полученного в верхнем элементе магазина, на нетерминал в левой части данного правила грамматики.

3. Команды проверки на завершение разбора:

$$p_0, \varepsilon, SB_0 \rightarrow f, B_0.$$

Разбор завершается, если в магазине остались аксиома и маркер дна магазина, а входная цепочка полностью рассмотрена.

Подадим на вход автомата цепочку $a/(a+a)$ и выполним разбор. Процесс разбора представлен в таблице 5.

Таблица 5

[illegible]

Состоя- ние p	p_0	p_0	p_0	p_0	p_0	p_0	p_0	p_0	p_0	p_0	p_0	p_0	p_0	p_0	p_0	p_0	f
--------------------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-----

5.4.2. Нисходящий разбор в МП-автомате

На любом шаге нисходящего разбора должно применяться какое-либо правило. В начальный момент таким нетерминалом является аксиома. МП-автомат, выполняющий нисходящий разбор, работает по следующему алгоритму:

1. в начальный момент времени в магазин заносится аксиома: $p_0, \varepsilon, \varepsilon \rightarrow p_1, S$;
2. для каждого правила $A \rightarrow \phi \in P$ нетерминал в верхушке магазина заменяется на правую часть правила с помощью команды: $p_1, \varepsilon, A \rightarrow p_1, \phi$;
3. для каждого терминала $a \in V_T$ выполняется сравнение символа на входной ленте с символом в верхушке магазина и его поглощение: $p_1, a, a \rightarrow p_1, \varepsilon$;
4. разбор заканчивается по команде: $p_1, \varepsilon, B_0 \rightarrow f, B_0$.

Для грамматики, рассмотренной в предыдущем примере, разбор той же входной цепочки по нисходящей стратегии будет выполняться посредством следующего множества команд:

1. команда занесения аксиомы в магазин:
 $p_0, \varepsilon, \varepsilon \rightarrow p_0, S$;
2. команды замены нетерминала правой частью правила:
 $p_1, \varepsilon, S \rightarrow p_1, S+A$
 $p_1, \varepsilon, S \rightarrow p_1, S/A$
 $p_1, \varepsilon, S \rightarrow p_1, A$
 $p_1, \varepsilon, A \rightarrow p_1, a$
 $p_1, \varepsilon, A \rightarrow p_1, (S)$;
3. команды сравнения и поглощения символа с входной ленты и символа в верхушке магазина:
 $p_1, a, a \rightarrow p_1, \varepsilon$
 $p_1, +, + \rightarrow p_1, \varepsilon$
 $p_1, /, / \rightarrow p_1, \varepsilon$
 $p_1, (, (\rightarrow p_1, \varepsilon$
 $p_1,),) \rightarrow p_1, \varepsilon$;
- 4) команда завершения разбора:
 $p_1, \varepsilon, B_0 \rightarrow f, B_0$.

Процесс разбора цепочки представлен в таблице 6.

Таблица 6

Вход	$a \quad / \quad (\quad a \quad + \quad a \quad)$																
магазин			S	a	a			$($		S	A	a					
		S	$/$	$/$	$/$	$/$		S	S	$+$	$+$	$+$	$+$				
		A	A	A	A	A	A	$)$	$)$	A	A	A	A	A	a		
	B_0	B	B	B	B	B	B	B	B	B	B	B	B	B	B	B	B
		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Состояние p	p_0	p_1	p_1	p_1	p_1	p_1	p_1	p_1	p_1	p_1	p_1	p_1	p_1	p_1	p_1	p_1	f

5.5. Назначение МП-автоматов

Рассмотренные выше МП-автоматы работают недетерминированно, то есть если цепочка принадлежит языку, порождаемому заданной грамматикой, то какой-то из вариантов функционирования автомата осуществит правильный разбор. Если цепочка не принадлежит языку, то никакой из вариантов разбора не приведет к цели.

Отсутствие детерминированного эквивалентного автомата для произвольной КС-грамматики означает невозможность построения универсальной простой однократной программы синтаксического анализа. Поэтому для эффективного разбора необходимо выделять специальные классы КС-грамматик, удовлетворяющие требованиям конкретных типов анализаторов.

Если требуется выполнить разбор для произвольной КС-грамматики, то придется использовать детерминированную программную модель недетерминированного МП-автомата.

5.6. Понятие преобразователей

Автоматы с выходом называются *преобразователями*. В зависимости от вида функции, отображающей множество состояний и входных символов в множество выходных символов и новых состояний, а также от типа рабочей ленты различают разные виды преобразователей. Рассмотрим конечные автоматы-преобразователи.

Конечным преобразователем называется шестерка вида

$P = (K, X, Y, f, g, q_0)$, где

K - конечное множество состояний;

X - входной алфавит;

Y - выходной алфавит;

f - функция переходов;

g - функция выходов;

q_0 - начальное состояние.

Типы отображений f и g определяют различные виды автоматов. Если g – отображение $K \cdot X$ в Y , то конечный преобразователь называется синхронным. В общем случае это отображение имеет вид $K \cdot X \rightarrow Y^*$.

Пусть $P = (K, X, Y, f, g, q_0)$ – конечный преобразователь. Тогда отображение $S(x) = g(q_0, x)$, определенное для любой цепочки $x \in X^*$, называется конечным преобразователем.

Для того чтобы выходную цепочку y можно было считать переводом входной цепочки x , цепочка x должна перевести преобразователь из начального состояния в заключительное.

5.7. Автоматы Мили и Мура

Автоматы *Мили* и *Мура* являются неинициальными автоматами. В отображении $S(x) = g(q_0, x)$ зафиксируем начальное состояние q_0 , в котором автомат находится в начальный момент времени. Оно существенно влияет на процесс конечного преобразования, т.к. определяет не только результирующую цепочку, но и множество входных цепочек.

Рассмотрим поведение инициальных автоматов, которые могут начинать работать из любого указанного состояния. Такой автомат получает на вход одну цепочку бесконечной длины и перерабатывает её. Реакция такого преобразователя на определенные воздействия непредсказуема, если неизвестно его начальное состояние. Поэтому необходимо решить две задачи, имеющие важное практическое значение:

1. определение того состояния автомата, в котором он находится в момент, начиная с которого исследуется его поведение;
2. распознавание конечного состояния, в которое перешел автомат после завершения испытательной операции. Это состояние будет начальным для следующей серии испытаний.

Эти задачи анализа получили название экспериментов по распознаванию состояния.

5.7.1. Автомат Мили

Автомат Мили – это пятерка вида $M = (K, X, Y, f, g)$, где:

K – множество состояний автомата;

X – входной алфавит;

Y – выходной алфавит;

f – функция переходов (отображение $K \cdot X \rightarrow K$);

g – функция выходов (отображение $K \cdot X \rightarrow Y$).

Как и любой другой автомат, автомат Мили можно представить в виде таблицы или графа. В графе переходов автомата Мили на дугах указываются через символ «/» входные и выходные символы. Таблица

переходов состоит из двух частей: в левой части записываются значения функции выходов, в правой части - значения функции переходов.

Пример. Построим преобразователь, который распознает арифметические выражения, порожаемые грамматикой:

$$S \rightarrow a+S \mid a-S \mid +S \mid -S \mid a$$

и устраняет из этих выражений избыточные унарные операции. Например, выражение $-a + -a - + -a$ он переведет в $-a -a +a$. Во входном языке символ a представляет идентификатор и перед идентификатором допускается произвольная последовательность знаков унарных операции $+$ и $-$. Заметим, что входной язык является регулярным множеством.

Пусть $M = (K, X, Y, f, g)$, где

$$K = \{q_0, q_1, q_2, q_3, q_4\},$$

$$X = \{a, +, -\},$$

$$Y = X.$$

Преобразователь M (рис. 10) начинает работу в состоянии q_0 и, чередуя состояния q_0 и q_4 на входном символе « $-$ », определяет, четное или нечетное число знаков $-$ предшествует первому символу a . Когда появляется a , преобразователь M переходит в состояние q_1 , допуская вход, и выдает a или $-a$ в зависимости от того, четно или нечетно число появившихся минусов. Для следующих символов a он подсчитывает, четно или нечетно число предшествующих минусов, с помощью состояний q_2 и q_3 . Единственное различие между парами q_2, q_3 и q_0, q_4 состоит в том, что если символу a предшествует четное число минусов, то первая из них выдает $+a$, а не только a .

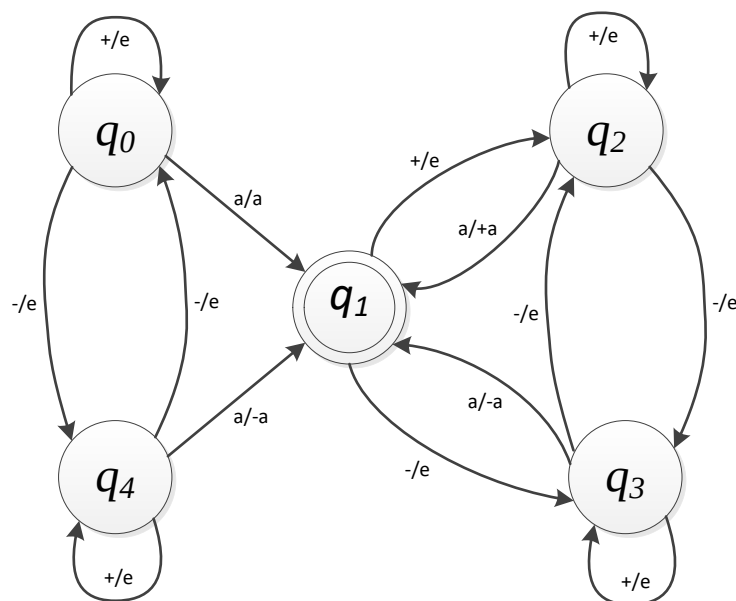


Рис. 10. Граф автомата Мили, распознающего простые арифметические выражения

Таблица переходов автомата выглядит следующим образом:

Таблица 7

Таблица переходов автомата Мили

	Y			K		
K\X	a	+	-	a	+	-
q ₀	a	ε	ε	q ₁	q ₀	q ₄
q ₁	-	ε	ε	-	q ₂	q ₃
q ₂	+a	ε	ε	q ₁	q ₂	q ₃
q ₃	-a	ε	ε	q ₁	q ₃	q ₂
q ₄	-a	ε	ε	q ₁	q ₄	q ₀

5.7.2. Автомат Мура

Автомат Мура - это «пятерка» вида $U = (K_1, X, Y, f_1, h)$, где:

K_1 - множество состояний автомата;

X - входной алфавит;

Y - выходной алфавит;

f_1 - функция переходов (отображение $K \cdot X \rightarrow K$);

h - функция выходов (отображение $K \cdot X \rightarrow Y$).

При представлении автомата Мура графом дуги помечаются символами входного алфавита, а каждая вершина графа - состоянием и символом выходного алфавита.

При формальном сравнении определений автоматов Мили и Мура может показаться, что автомат Мура может быть задан как независимый от входного состояния автомат Мили, т.е. такой автомат Мили, выходная функция которого удовлетворяет следующим условиям:

$$a \in X, \forall b \in X, \forall z \in Z (g(z, a) = g(z, b)). \quad (19)$$

Но это не соответствует способу функционирования автоматов Мура в соответствии с введенным определением.

В автомате Мура реализована иная временная связь между переходами из одного состояния в другое и выходом, по сравнению с автоматом Мили, у которого выход, соответствующий некоторому входу и определенному состоянию, порождается во время перехода автомата в следующее состояние. У автомата Мура сначала порождается выход, а потом - переход в следующее состояние, причем выход определяется только состоянием автомата.